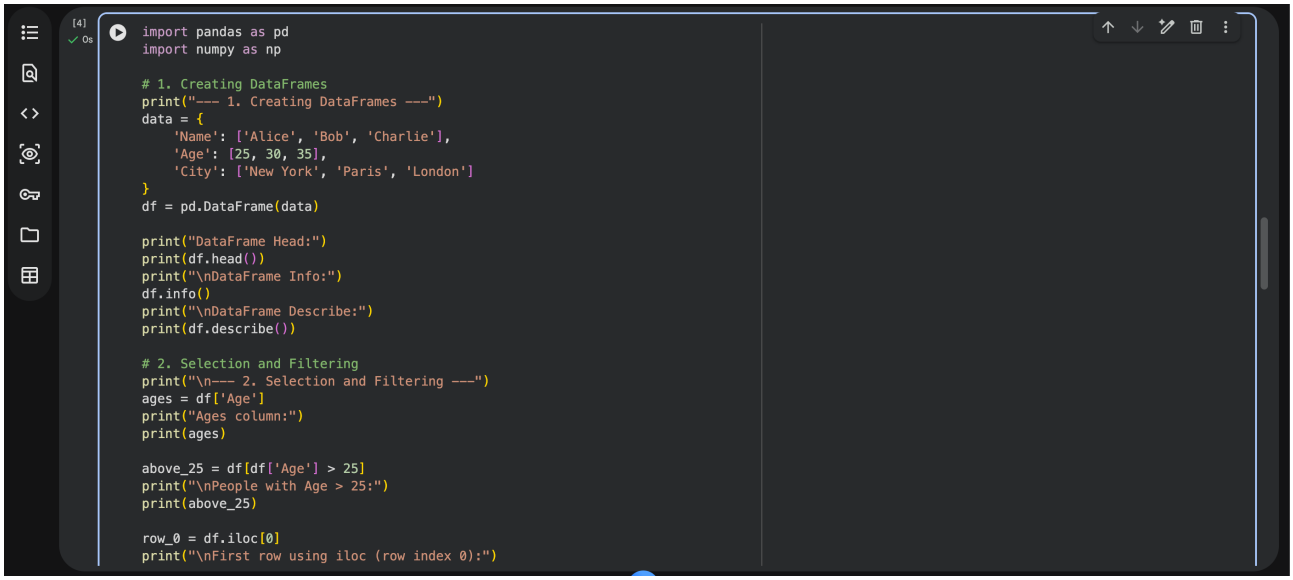
Experiment 2: Data Cleaning and Handling Missing Values Using numpy

Aim:

To identify and handle missing values in a Pandas DataFrame using `isnull()`, `dropna()`, and `fillna()` methods.

Algorithm:

1. Import the Pandas and NumPy libraries.
2. Create a sample dataset containing Name, Age, and City columns with some missing values represented by `NaN`.
3. Convert the dataset into a DataFrame using `pd.DataFrame()`.
4. Display the original DataFrame.
5. Identify and count missing values in each column using `df.isnull().sum()`.
6. Remove rows containing missing values using `df.dropna()`.
7. Replace all missing values with `0` using `df.fillna(0)`.
8. Create a copy of the original DataFrame and replace missing Age values with the mean Age using `fillna()`.
9. Create another copy and replace missing City values with `"Unknown"`.
10. Display the results of all data-cleaning operations.



```
[4]
✓ 0s
import pandas as pd
import numpy as np

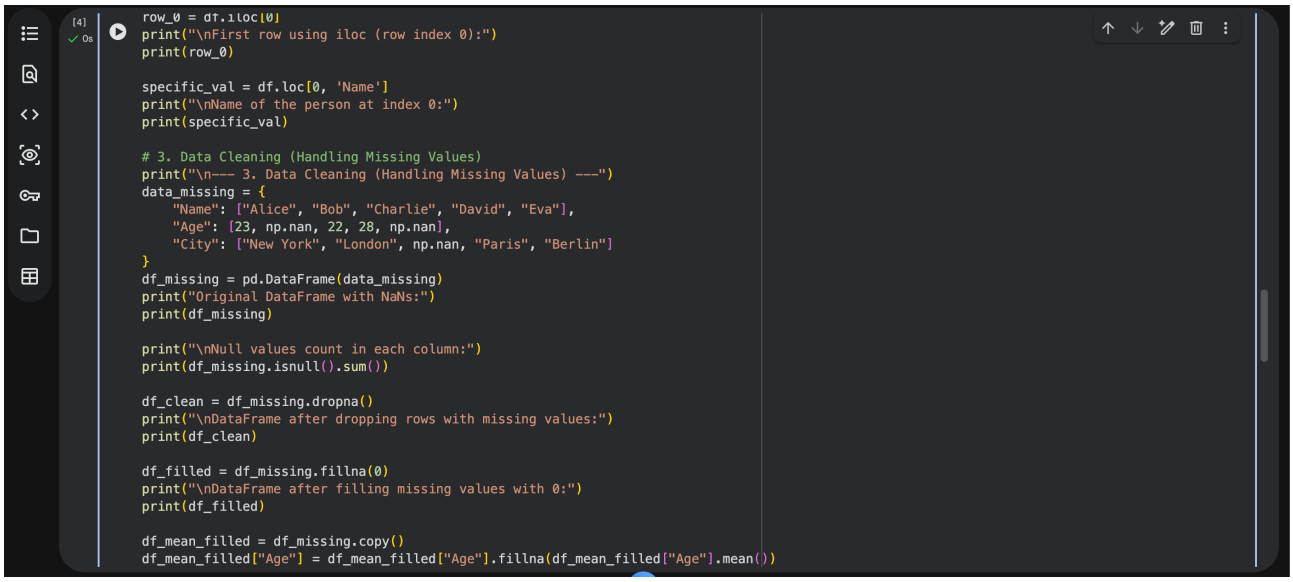
# 1. Creating DataFrames
print("--- 1. Creating DataFrames ---")
data = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35],
    'City': ['New York', 'Paris', 'London']
}
df = pd.DataFrame(data)

print("DataFrame Head:")
print(df.head())
print("\nDataFrame Info:")
df.info()
print("\nDataFrame Describe:")
print(df.describe())

# 2. Selection and Filtering
print("\n--- 2. Selection and Filtering ---")
ages = df['Age']
print("Ages column:")
print(ages)

above_25 = df[df['Age'] > 25]
print("\nPeople with Age > 25:")
print(above_25)

row_0 = df.iloc[0]
print("\nFirst row using iloc (row index 0):")
```



```
[4]
✓ 0s
row_0 = df.iloc[0]
print("\nFirst row using iloc (row index 0):")
print(row_0)

specific_val = df.loc[0, 'Name']
print("\nName of the person at index 0:")
print(specific_val)

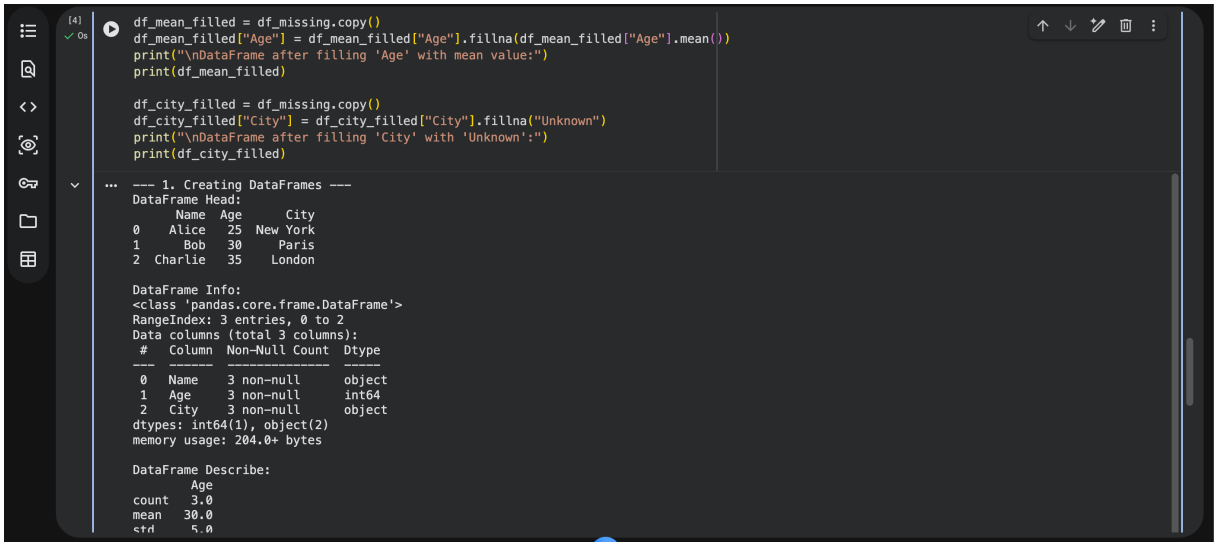
# 3. Data Cleaning (Handling Missing Values)
print("\n--- 3. Data Cleaning (Handling Missing Values) ---")
data_missing = {
    "Name": ["Alice", "Bob", "Charlie", "David", "Eva"],
    "Age": [23, np.nan, 22, 28, np.nan],
    "City": ["New York", "London", np.nan, "Paris", "Berlin"]
}
df_missing = pd.DataFrame(data_missing)
print("Original DataFrame with NaNs:")
print(df_missing)

print("\nNull values count in each column:")
print(df_missing.isnull().sum())

df_clean = df_missing.dropna()
print("\nDataFrame after dropping rows with missing values:")
print(df_clean)

df_filled = df_missing.fillna(0)
print("\nDataFrame after filling missing values with 0:")
print(df_filled)

df_mean_filled = df_missing.copy()
df_mean_filled["Age"] = df_mean_filled["Age"].fillna(df_mean_filled["Age"].mean())
```



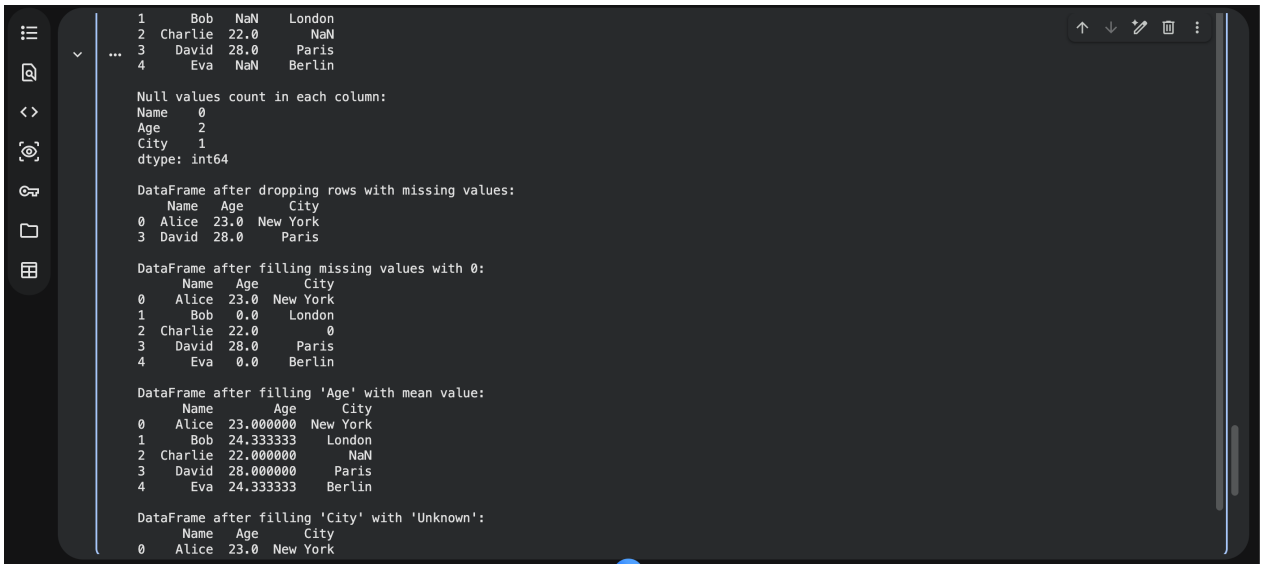
```
[4]
✓ 0s
df_mean_filled = df_missing.copy()
df_mean_filled["Age"] = df_mean_filled["Age"].fillna(df_mean_filled["Age"].mean())
print("\nDataFrame after filling 'Age' with mean value:")
print(df_mean_filled)

df_city_filled = df_missing.copy()
df_city_filled["City"] = df_city_filled["City"].fillna("Unknown")
print("\nDataFrame after filling 'City' with 'Unknown':")
print(df_city_filled)
```

```
--- 1. Creating DataFrames ---
DataFrame Head:
      Name  Age      City
0    Alice   25  New York
1      Bob   30     Paris
2  Charlie   35    London

DataFrame Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3 entries, 0 to 2
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Name    3 non-null      object
 1   Age     3 non-null      int64
 2   City    3 non-null      object
dtypes: int64(1), object(2)
memory usage: 204.0+ bytes

DataFrame Describe:
             Age
count   3.0
mean   30.0
std     5.0
```



```
1      Bob   NaN    London
2  Charlie  22.0       NaN
3    David  28.0     Paris
4      Eva   NaN    Berlin

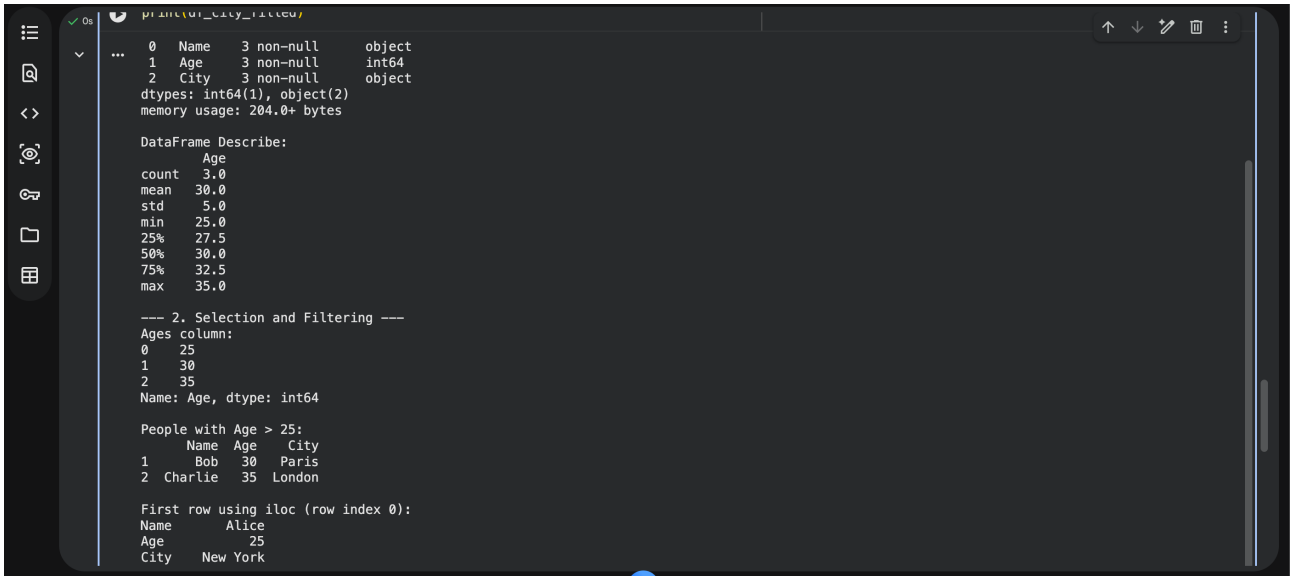
Null values count in each column:
Name    0
Age     2
City    1
dtype: int64

DataFrame after dropping rows with missing values:
    Name   Age      City
0  Alice  23.0  New York
3  David  28.0     Paris

DataFrame after filling missing values with 0:
      Name   Age      City
0    Alice  23.0  New York
1      Bob   0.0    London
2  Charlie  22.0         0
3    David  28.0     Paris
4      Eva   0.0    Berlin

DataFrame after filling 'Age' with mean value:
      Name        Age      City
0    Alice  23.000000  New York
1      Bob  24.333333    London
2  Charlie  22.000000       NaN
3    David  28.000000     Paris
4      Eva  24.333333    Berlin

DataFrame after filling 'City' with 'Unknown':
      Name   Age      City
0    Alice  23.0  New York
```



Result:

Missing values were identified and handled successfully by removing incomplete rows, replacing missing values with zero, filling missing Age values with the column mean, and replacing missing City values with `"Unknown"`.